

OS Lab Code Snippets

written by Obidit—220041154

22nd January 2026

1 Basics and Race Condition

Threads and Simple Routine:

simpleRoutine.c:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* routine(void* arg) {
    printf("Test for threads\n");
    sleep(3);
    printf("Ending Thread\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, routine, NULL);
    pthread_create(&t2, NULL, routine, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Below is an example demonstrating a race condition using threads in C:

raceCondition.c:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0; // Shared resource
```

```
void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++; // Critical section
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    // two threads are running concurrently - main + increment
    // thread (t1)
    pthread_create(&t2, NULL, increment, NULL);
    // now three threads are running concurrently (main, t1, t2)

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);
    return 0;
}
```

2 Mutex and Semaphore

Mutex Example using pthreads to solve race condition:

mutexExample.c:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0; // Shared resource
pthread_mutex_t lock; // Mutex

void* increment(void* arg) {
```

```

    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock); // Enter critical region
        counter++;
        pthread_mutex_unlock(&lock); // Exit critical region
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL); // Initialize mutex

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock); // Destroy mutex

    printf("Final counter value: %d\n", counter);
    return 0;
}

```

semaphoreExample.c:

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h> // Required for semaphores

int counter = 0; // Shared resource

sem_t sem; // Semaphore

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        sem_wait(&sem); // Enter critical section
        counter++;      // Critical section
        sem_post(&sem); // Exit critical section
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    // Initialize semaphore with value 1 (for mutual exclusion)
    sem_init(&sem, 0, 1); // Initialize semaphore

```

```

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem); // Cleanup

    printf("Final counter value: %d\n", counter);
    return 0;
}

```

3 Solution of Practise Tasks

hotSwappableConfiguration.c:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

// Global configuration pointer
int* active_config = NULL;

// Single mutex (only one allowed)
pthread_mutex_t config_mutex = PTHREAD_MUTEX_INITIALIZER;

/* Reader thread */
void* reader(void* arg) {
    int id = *(int*)arg;

    while (1) {
        pthread_mutex_lock(&config_mutex);

        // Guaranteed safe access
        if (active_config != NULL) {
            printf("Reader %d reads config value: %d\n",
                id, *active_config);
        }

        pthread_mutex_unlock(&config_mutex);

        sleep(1);
    }
    return NULL;
}

/* Updater thread */

```

```

void* updater(void* arg) {
    int value = 100;

    while (1) {
        pthread_mutex_lock(&config_mutex);

        // Free old configuration
        free(active_config);

        // Allocate new configuration
        active_config = malloc(sizeof(int));

        // Invariant: active_config != NULL
        *active_config = value++;

        printf("Updater installed new config value: %d\n",
            *active_config);

        pthread_mutex_unlock(&config_mutex);

        sleep(3);
    }
    return NULL;
}

int main() {
    pthread_t r1, r2, r3, up;
    int id1 = 1, id2 = 2, id3 = 3;

    // Initial configuration (invariant must hold)
    active_config = malloc(sizeof(int));
    *active_config = 42;

    pthread_create(&r1, NULL, reader, &id1);
    pthread_create(&r2, NULL, reader, &id2);
    pthread_create(&r3, NULL, reader, &id3);
    pthread_create(&up, NULL, updater, NULL);

    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    pthread_join(r3, NULL);
    pthread_join(up, NULL);

    return 0;
}

```

3.1 diningPhilosopher.c

```
#include <stdio.h>
```

```

#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

// One counting semaphore for room capacity
sem_t room;

// One semaphore per fork
sem_t forks[N];

void* philosopher(void* arg) {
    int id = *(int*)arg;
    int left = id;
    int right = (id + 1) % N;

    while (1) {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        // Enter the room (max 4 philosophers)
        sem_wait(&room);

        // Pick up forks
        sem_wait(&forks[left]);
        sem_wait(&forks[right]);

        printf("Philosopher %d is eating\n", id);
        sleep(2);

        // Put down forks
        sem_post(&forks[right]);
        sem_post(&forks[left]);

        // Leave the room
        sem_post(&room);
    }
    return NULL;
}

int main() {
    pthread_t phil[N];
    int id[N];

    // Initialize room semaphore to 4
    sem_init(&room, 0, 4);

    // Initialize forks
    for (int i = 0; i < N; i++) {

```

```

    sem_init(&forks[i], 0, 1);
    id[i] = i;
}

// Create philosopher threads
for (int i = 0; i < N; i++) {
    pthread_create(&phil[i], NULL, philosopher, &id[i]);
}

// Join threads (never ends)
for (int i = 0; i < N; i++) {
    pthread_join(phil[i], NULL);
}

return 0;
}

```

histogram.c:

```

#include <stdio.h>
#include <pthread.h>

#define HIST_SIZE 10
#define N 9
#define THREADS 2

int input[N] = {1, 3, 2, 5, 3, 7, 8, 3, 1};
int histogram[HIST_SIZE] = {0};

pthread_mutex_t mutex;

// Structure to pass range info to threads
typedef struct {
    int start;
    int end;
} ThreadData;

void* compute_histogram(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    int local_hist[HIST_SIZE] = {0};

    // Step 1: Compute local histogram
    for (int i = data->start; i <= data->end; i++) {
        int val = input[i];
        local_hist[val]++;
    }

    // Step 2: Merge into global histogram (critical section)
    pthread_mutex_lock(&mutex);

```

```

    for (int i = 0; i < HIST_SIZE; i++) {
        histogram[i] += local_hist[i];
    }
    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main() {
    pthread_t threads[THREADS];
    ThreadData ranges[THREADS];

    pthread_mutex_init(&mutex, NULL);

    // Compute ranges: split N elements as evenly as possible
    // across THREADS
    int base = N / THREADS; // minimum chunk size
    int extra = N % THREADS; // first 'extra' threads handle one
    // more element
    int cursor = 0;
    for (int i = 0; i < THREADS; i++) {
        int count = base + (i < extra ? 1 : 0);
        ranges[i].start = cursor;
        ranges[i].end = cursor + count - 1; // inclusive end
        cursor += count;
    }

    // Create threads
    for (int i = 0; i < THREADS; i++) {
        pthread_create(&threads[i], NULL, compute_histogram, &
            ranges[i]);
    }

    // Join threads
    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&mutex);

    // Print histogram
    int sum = 0;
    printf("Final Histogram:\n");
    for (int i = 0; i < HIST_SIZE; i++) {
        printf("histogram[%d] = %d\n", i, histogram[i]);
        sum += histogram[i];
    }
    printf("Sum of histogram values = %d\n", sum);
    return 0;
}

```

h20Bond.c:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t hReady;
sem_t hQueue;
sem_t oQueue;
sem_t barrier;

void* hydrogen(void* arg) {
    printf("H Ready\n");

    // Signal availability
    sem_post(&hReady);

    // Wait until selected for bonding
    sem_wait(&hQueue);

    // Barrier: wait for other atoms
    sem_wait(&barrier);

    printf("H Bonded\n");
    return NULL;
}

void* oxygen(void* arg) {
    printf("O Ready\n");

    // Wait for two hydrogen atoms
    sem_wait(&hReady);
    sem_wait(&hReady);

    // Release exactly two hydrogens
    sem_post(&hQueue);
    sem_post(&hQueue);

    // Barrier release: 3 atoms total
    sem_post(&barrier);
    sem_post(&barrier);
    sem_post(&barrier);

    printf("Bond Formed!\n");
    return NULL;
}

int main() {
    pthread_t h[4], o[2];
```

```
    sem_init(&hReady, 0, 0);
    sem_init(&hQueue, 0, 0);
    sem_init(&oQueue, 0, 0);
    sem_init(&barrier, 0, 0);

    // Create Hydrogen threads
    for (int i = 0; i < 4; i++) {
        pthread_create(&h[i], NULL, hydrogen, NULL);
        sleep(1);
    }

    // Create Oxygen threads
    for (int i = 0; i < 2; i++) {
        pthread_create(&o[i], NULL, oxygen, NULL);
        sleep(2);
    }

    for (int i = 0; i < 4; i++)
        pthread_join(h[i], NULL);

    for (int i = 0; i < 2; i++)
        pthread_join(o[i], NULL);

    return 0;
}
```

multiAccountTransfer.c:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Shared accounts
int A = 1000, B = 1000, C = 1000;

// Mutexes for each account
pthread_mutex_t mutexA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutexB = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutexC = PTHREAD_MUTEX_INITIALIZER;

/*
Thread T1 Transfers 100 from A to B 10 times.
Thread T2 Transfers 50 from B to C 20 times.
Thread T3 Transfers 25 from C to A 40 times.
*/

// Utility function to print balances
void print_balances(const char* msg) {
```

```

    printf("%s | A=%d B=%d C=%d | Total=%d\n",
           msg, A, B, C, A + B + C);
}

// Thread T1: A -> B (100) x10
void* thread1(void* arg) {
    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutexA);
        pthread_mutex_lock(&mutexB);

        if (A >= 100) {
            A -= 100;
            B += 100;
            print_balances("T1: A -> B (100)");
        }

        pthread_mutex_unlock(&mutexB);
        pthread_mutex_unlock(&mutexA);

        usleep(100000);
    }
    return NULL;
}

// Thread T2: B -> C (50) x20
void* thread2(void* arg) {
    for (int i = 0; i < 20; i++) {
        pthread_mutex_lock(&mutexB);
        pthread_mutex_lock(&mutexC);

        if (B >= 50) {
            B -= 50;
            C += 50;
            print_balances("T2: B -> C (50)");
        }

        pthread_mutex_unlock(&mutexC);
        pthread_mutex_unlock(&mutexB);

        usleep(100000);
    }
    return NULL;
}

// Thread T3: C -> A (25) x40
void* thread3(void* arg) {
    for (int i = 0; i < 40; i++) {
        // IMPORTANT: lock A first, then C (global order)
        pthread_mutex_lock(&mutexA);
        pthread_mutex_lock(&mutexC);

```

```

        if (C >= 25) {
            C -= 25;
            A += 25;
            print_balances("T3: C -> A (25)");
        }

        pthread_mutex_unlock(&mutexC);
        pthread_mutex_unlock(&mutexA);

        usleep(100000);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;

    print_balances("Initial");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
    pthread_create(&t3, NULL, thread3, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    print_balances("Final");

    return 0;
}

```

transactionServer.c

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_CLIENTS 6
#define SERVER_CAPACITY 3

sem_t serverSlots;
pthread_mutex_t lock;
int status[NUM_CLIENTS] = {0};
int running = 1;

void* client_thread(void* arg) {

```

```

int id = *(int*)arg;

// Request server access
sem_wait(&serverSlots);

// Mark status = True
pthread_mutex_lock(&lock);
status[id] = 1;
pthread_mutex_unlock(&lock);

// Simulate processing time
sleep(1);

// Mark status = False
pthread_mutex_lock(&lock);
status[id] = 0;
pthread_mutex_unlock(&lock);

// Release slot
sem_post(&serverSlots);

return NULL;
}

void* monitor_thread(void* arg) {
    while (running) {
        pthread_mutex_lock(&lock);
        for (int i = 0; i < NUM_CLIENTS; i++) {
            printf("C%d=%s ", i + 1, status[i] ? "True" : "False
                ");
        }
        printf("\n");
        pthread_mutex_unlock(&lock);

        usleep(500000); // print every 0.5 sec
    }
    return NULL;
}

int main() {
    pthread_t clients[NUM_CLIENTS], monitor;
    int ids[NUM_CLIENTS];

    sem_init(&serverSlots, 0, SERVER_CAPACITY);
    pthread_mutex_init(&lock, NULL);

    // Start monitor
    pthread_create(&monitor, NULL, monitor_thread, NULL);

    // Start clients

```

```

for (int i = 0; i < NUM_CLIENTS; i++) {
    ids[i] = i;
    pthread_create(&clients[i], NULL, client_thread, &ids[i]
        );
    usleep(100000); // small stagger to simulate near-
        simultaneous arrival
}

// Wait for clients to finish
for (int i = 0; i < NUM_CLIENTS; i++)
    pthread_join(clients[i], NULL);

running = 0;
pthread_join(monitor, NULL);

sem_destroy(&serverSlots);
pthread_mutex_destroy(&lock);

return 0;
}

```

navigationLaunchSystem.c:

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem_nav; // Signal from Fuel -> Navigation
sem_t sem_launch; // Signal from Navigation -> Launch

void* fuel_thread(void* arg) {
    sleep(1); // simulate work
    printf("[Fuel Thread] Fuel system activated. Signal sent.\n"
        );
    sem_post(&sem_nav); // allow navigation to start
    return NULL;
}

void* navigation_thread(void* arg) {
    printf("[Navigation Thread] Waiting for fuel system...\n");
    sem_wait(&sem_nav); // wait for fuel
    sleep(1);
    printf("[Navigation Thread] Navigation system online. Signal
        sent.\n");
    sem_post(&sem_launch); // allow launch
    return NULL;
}

```

```

void* launch_thread(void* arg) {
    printf("[Launch Thread] Waiting for navigation system...\n");
    ;
    sem_wait(&sem_launch); // wait for navigation
    sleep(1);
    printf("[Launch Thread] Launch sequence initiated!\n");
    return NULL;
}

int main() {
    pthread_t fuel, navigation, launch;

    // Initialize semaphores to 0 (locked)
    sem_init(&sem_nav, 0, 0);
    sem_init(&sem_launch, 0, 0);

    // Create threads in ANY order (result will still be
    sequenced)
    pthread_create(&launch, NULL, launch_thread, NULL);
    pthread_create(&navigation, NULL, navigation_thread, NULL);
    pthread_create(&fuel, NULL, fuel_thread, NULL);

    // Wait for all threads
    pthread_join(fuel, NULL);
    pthread_join(navigation, NULL);
    pthread_join(launch, NULL);

    sem_destroy(&sem_nav);
    sem_destroy(&sem_launch);

    return 0;
}

```

readersWriters.c:

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int data = 0;
int read_count = 0;

sem_t mutex; // protects read_count
sem_t write_block; // blocks writers/readers

void* reader(void* arg) {
    int id = *(int*)arg;
    printf("[Reader %d] Waiting to read...\n", id);

```

```

// Entry section
sem_wait(&mutex);
read_count++;
if (read_count == 1) {
    sem_wait(&write_block); // first reader blocks writers
}
sem_post(&mutex);

// Critical section (reading)
printf("[Reader %d] Reading data: %d\n", id, data);
sleep(1);

// Exit section
sem_wait(&mutex);
read_count--;
if (read_count == 0) {
    sem_post(&write_block); // last reader allows writers
}
sem_post(&mutex);

return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;
    printf("[Writer %d] Waiting to write...\n", id);

    sem_wait(&write_block); // Writer gets exclusive access

    // Critical section (writing)
    printf("[Writer %d] Writing data...\n", id);
    data++;
    sleep(1);
    printf("[Writer %d] Updated data to %d\n", id, data);

    sem_post(&write_block);

    return NULL;
}

int main() {
    pthread_t r[3], w[1];
    int r_id[3] = {1, 2, 3};
    int w_id[1] = {1};

    sem_init(&mutex, 0, 1);
    sem_init(&write_block, 0, 1);

    // Start threads in mixed order to simulate randomness

```



```

pthread_create(&r[0], NULL, reader, &r_id[0]);
pthread_create(&r[1], NULL, reader, &r_id[1]);
pthread_create(&w[0], NULL, writer, &w_id[0]);
sleep(1); // delay to simulate next arrivals
pthread_create(&r[2], NULL, reader, &r_id[2]);

for (int i = 0; i < 3; i++)
    pthread_join(r[i], NULL);
pthread_join(w[0], NULL);

sem_destroy(&mutex);
sem_destroy(&write_block);

return 0;
}

```

priorityLogging.c:

```

#include <stdio.h>
#include <pthread.h>
#include <string.h>

#define MAX 10

pthread_mutex_t mutex;

char high_buffer[MAX][50];
char low_buffer[MAX][50];
int high_count = 0, low_count = 0;

void* log_thread(void* arg) {
    char* msg = (char*)arg;
    pthread_mutex_lock(&mutex);

    if (msg[0] == 'H') { // High priority message flag
        strcpy(high_buffer[high_count++], msg + 2); // skip "H:"
    } else {
        strcpy(low_buffer[low_count++], msg + 2); // skip "L:"
    }

    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3, t4;

    pthread_mutex_init(&mutex, NULL);

```

```

// Priority-tagged messages as per input
pthread_create(&t1, NULL, log_thread, "L:Network OK");
pthread_create(&t2, NULL, log_thread, "H:Disk Failure");
pthread_create(&t3, NULL, log_thread, "L>User Login");
pthread_create(&t4, NULL, log_thread, "H:Power Surge");

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

// Buffer dump respecting priority
printf("----- Buffer Dump -----\\n");
for (int i = 0; i < high_count; i++)
    printf("[HIGH] %s\\n", high_buffer[i]);

for (int i = 0; i < low_count; i++)
    printf("[LOW] %s\\n", low_buffer[i]);

pthread_mutex_destroy(&mutex);
return 0;
}

```

parallelHistogram.c:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define SUB_ARRAY_COUNT 4
#define SUB_ARRAY_DIM 10
#define RANGE 10

int data_array[SUB_ARRAY_COUNT][SUB_ARRAY_DIM][SUB_ARRAY_DIM][
    SUB_ARRAY_DIM];
int global_histogram[RANGE] = {0};

pthread_mutex_t mutex;

void init_data_array() {
    srand(0);
    for (int i = 0; i < SUB_ARRAY_COUNT; i++) {
        for (int j = 0; j < SUB_ARRAY_DIM; j++) {
            for (int k = 0; k < SUB_ARRAY_DIM; k++) {
                for (int l = 0; l < SUB_ARRAY_DIM; l++) {
                    data_array[i][j][k][l] = rand() % RANGE;
                }
            }
        }
    }
}

```

```

    }
}

void* compute_histogram(void* arg) {
    int index = *(int*)arg;

    int local_histogram[RANGE] = {0};

    for (int j = 0; j < SUB_ARRAY_DIM; j++) {
        for (int k = 0; k < SUB_ARRAY_DIM; k++) {
            for (int l = 0; l < SUB_ARRAY_DIM; l++) {
                int value = data_array[index][j][k][l];
                local_histogram[value]++;
            }
        }
    }

    pthread_mutex_lock(&mutex);
    for (int i = 0; i < RANGE; i++) {
        global_histogram[i] += local_histogram[i];
    }
    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main() {
    pthread_t threads[SUB_ARRAY_COUNT];
    int indices[SUB_ARRAY_COUNT];

    pthread_mutex_init(&mutex, NULL);

    init_data_array();

    for (int i = 0; i < SUB_ARRAY_COUNT; i++) {
        indices[i] = i;
        pthread_create(&threads[i], NULL, compute_histogram, &
            indices[i]);
    }

    for (int i = 0; i < SUB_ARRAY_COUNT; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&mutex);

    int total = 0;
    for (int i = 0; i < RANGE; i++) {
        total += global_histogram[i];
    }
}

```

```

printf("Total histogram count: %d\n", total);
printf("Expected count: %d\n",
        SUB_ARRAY_COUNT * SUB_ARRAY_DIM * SUB_ARRAY_DIM *
        SUB_ARRAY_DIM);

return 0;
}

```

4 Bash Scripting

Every script needs a “Shebang” and proper execution permissions to be recognized by the system as an executable script.

4.1 Shebang and Execution

To run a script, you must provide the path to the interpreter and ensure the file has execute permissions.

```

#!/bin/bash
# This is a single-line comment
echo "Hello, World!" # Printing to the console

# To run this script:
# 1. chmod +x script.sh
# 2. ./script.sh

```

5 Variables

Variables in Bash are untyped by default. **Crucial Rule:** Do not put spaces around the = sign during assignment.

5.1 Assignment and Accessing Variables

Use double quotes to prevent word splitting and curly braces to clarify variable boundaries.

```

#!/bin/bash
# Assignment (No spaces!)
NAME="Gemini"
AGE=21
IS_BOT=true

# Accessing variables
echo "My name is $NAME"
echo "I am ${AGE} years old"

```

```
# Read-only variables
readonly SECRET="12345"

# Command Substitution (Saving command output to a variable)
CURRENT_DIR=$(pwd)
echo "You are currently in: $CURRENT_DIR"
```

6 Arithmetic Operations

Bash treats variables as strings by default, so you need specific syntax for math. The most common method is using Double Parentheses `$( ( ))`.

6.1 Integer and Floating-Point Math

While Bash handles integer division, external tools like `bc` are required for decimals.

```
#!/bin/bash
NUM1=10
NUM2=5

# Addition, Subtraction, Multiplication, Division
SUM=$((NUM1 + NUM2))
DIFF=$((NUM1 - NUM2))
PRODUCT=$((NUM1 * NUM2))
QUOTIENT=$((NUM1 / NUM2)) # Note: Bash only does integer division

echo "Sum: $SUM"
echo "Product: $PRODUCT"

# Increment / Decrement
((NUM1++))
((NUM2--))

# For floating-point (decimals), use 'bc'
PI=$(echo "scale=2; 22/7" | bc)
echo "Pi is approximately: $PI"
```

7 Tests and Conditions

Bash uses `if` statements and “tests” (usually wrapped in `[[ ]]`) to control logic flow.

7.1 Comparison Operators

The table below outlines the operators used for different data types.

Type	Operator	Meaning
String	<code>==</code> / <code>!=</code>	Equal / Not equal
Numeric	<code>-eq</code> / <code>-ne</code>	Equal / Not equal
Numeric	<code>-gt</code> / <code>-lt</code>	Greater than / Less than
File	<code>-e</code> / <code>-f</code>	Exists / Is a regular file

7.2 If / Else Blocks

Conditional blocks must be closed with the `fi` keyword.

```
#!/bin/bash
SCORE=85

if [[ $SCORE -ge 90 ]]; then
    echo "Grade: A"
elif [[ $SCORE -ge 80 ]]; then
    echo "Grade: B"
else
    echo "Grade: C"
fi
```

7.3 The “Ternary” Shortcut

Bash doesn’t have a built-in `?:` operator, but you can achieve the same result using logic gates `&&` (AND) and `||` (OR).

```
# Syntax: [[ condition ]] && value_if_true || value_if_false
USER_STATUS="online"

# Example:
[[ "$USER_STATUS" == "online" ]] && echo "User is active" ||
    echo "User is away"

# Storing it in a variable:
MESSAGE=$(( [ "$USER_STATUS" == "online" ] && echo "Welcome back" ||
    echo "Please log in." ))
echo "$MESSAGE"
```

8 Loops

Loops allow you to run a block of code multiple times. The `for` loop is best for lists, while the `while` loop is best for conditions.

```
#!/bin/bash

# --- For Loop: Iterating over a list ---
echo "Counting colors:"
```

```

for COLOR in "Red" "Green" "Blue"; do
    echo "Color is: $COLOR"
done

# --- For Loop: C-style range ---
for ((i=1; i<=3; i++)); do
    echo "Number: $i"
done

# --- While Loop: Run until a condition is false ---
COUNT=1
while [[ $COUNT -le 3 ]]; do
    echo "Iteration: $COUNT"
    ((COUNT++))
done

# --- Until Loop: Run until a condition becomes true ---
TIMER=3
until [[ $TIMER -eq 0 ]]; do
    echo "Countdown: $TIMER"
    ((TIMER--))
done

```

9 Arrays

Bash supports one-dimensional indexed arrays. Note that indices start at 0.

```

#!/bin/bash

# Define an array
FRUITS=("Apple" "Banana" "Cherry")

# Add an element
FRUITS+=("Date")

# Access specific element
echo "First fruit: ${FRUITS[0]}"

# Access ALL elements
echo "All fruits: ${FRUITS[@]}"

# Get the length of the array
echo "Total fruits: ${#FRUITS[@]}"

# Loop through an array
for FRUIT in "${FRUITS[@]}"; do
    echo "I like $FRUIT"
done

```

10 I/O Redirection

Standard Input (0), Output (1), and Error (2) are the three streams every command uses. Redirection allows you to send these streams to files or other commands.

Symbol	Description
>	Redirect stdout to a file (overwrites).
>>	Redirect stdout to a file (appends).
2>	Redirect stderr (errors) to a file.
&>	Redirect both stdout and stderr to a file.
<	Take stdin from a file instead of the keyboard.

Redirection Example

```

#!/bin/bash

# 1. Write output to a file (overwrite)
echo "Logging start time..." > log.txt

# 2. Append output to a file
echo "Session: $USER" >> log.txt

# 3. Redirect an error to a separate file
ls /root/secret 2> error.log

# 4. Silence a command (send output to the 'black hole')
ls /tmp > /dev/null 2>&1

# 5. Using a Pipe to filter data
# This lists files and only shows those ending in .txt
ls | grep ".txt"

```

11 Sample Bash Scripts

Here are practical Bash scripts demonstrating real-world logic and common use cases.

11.1 Array Math (Max, Min, Sum, Average)

This script prompts the user for a list of numbers and uses a loop to calculate statistics.

```

#!/bin/bash

echo "Enter numbers separated by spaces:"
read -a numbers

```

```

if [[ ${#numbers[@]} -eq 0 ]]; then
    echo "No numbers entered."
    exit 1
fi

sum=0
max=${numbers[0]}
min=${numbers[0]}

for n in "${numbers[@]"; do
    sum=$((sum + n))

    [[ $n -gt $max ]] && max=$n
    [[ $n -lt $min ]] && min=$n
done

# Calculate average (using 'bc' for decimal precision)
avg=$(echo "scale=2; $sum / ${#numbers[@]}" | bc)

echo "--- Results ---"
echo "Max: $max"
echo "Min: $min"
echo "Sum: $sum"
echo "Average: $avg"

```

11.2 Batch Rename (.h to .cpp)

This script iterates through all header files in the current directory and changes their extensions.

```

#!/bin/bash

count=0

for file in *.h; do
    # Check if files exist to avoid error if no .h files are found
    if [[ -e "$file" ]]; then
        # Use parameter expansion to strip .h and add .cpp
        mv "$file" "${file%.h}.cpp"
        ((count++))
    fi
done

echo "Renamed $count files."

```

11.3 File Type Identifier

This script uses file test operators to determine the nature of a path.

```

#!/bin/bash

read -p "Enter file or directory path: " path

if [[ ! -e "$path" ]]; then
    echo "Error: '$path' does not exist."
elif [[ -f "$path" ]]; then
    echo "'$path' is a regular file."
elif [[ -d "$path" ]]; then
    echo "'$path' is a directory."
else
    echo "'$path' exists but is a special file (link, socket, or device)."
fi

```

11.4 Permission Status (Positional Arguments)

This script checks the permissions for the current user. It uses the first argument passed to the script (\$1).

```

#!/bin/bash

# Check if an argument was provided
if [[ -z "$1" ]]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

FILE=$1

if [[ ! -e "$FILE" ]]; then
    echo "File not found."
    exit 1
fi

# Initialize status string
STATUS=""

[[ -r "$FILE" ]] && STATUS+="Readable " || STATUS+="Not-Readable "
[[ -w "$FILE" ]] && STATUS+="Writable " || STATUS+="Not-Writable "
[[ -x "$FILE" ]] && STATUS+="Executable " || STATUS+="Not-Executable "

echo "Permission status for '$FILE': $STATUS"

```

Quick Reference for File Tests

- `-e`: File exists.
- `-f`: Is a regular file.
- `-d`: Is a directory.
- `-r`: Is readable by you.
- `-w`: Is writable by you.
- `-x`: Is executable by you.

11.5 File Organizer by Extension

This script scans a directory and moves files into subfolders based on their file extensions (e.g., all `.jpg` files go to an Images folder).

```
#!/bin/bash

# Target directory (default to current)
DIR=${1:-.}

for file in "$DIR"/*; do
    # Skip if it's a directory
    if [[ -f "$file" ]]; then
        # Extract extension (e.g., ".jpg")
        ext="${file##*.}"

        # Create folder if it doesn't exist
        mkdir -p "$DIR/$ext"

        # Move file
        mv "$file" "$DIR/$ext/"
    fi
done

echo "Files organized by extension."
```

11.6 Automated Log Cleanup

To save disk space, this script finds and deletes files in a specific directory that are older than 30 days.

```
#!/bin/bash

LOG_DIR="/var/log/myapp"
DAYS=30

if [[ -d "$LOG_DIR" ]]; then
```

```
# find: search for files
# -mtime +30: modified more than 30 days ago
# -delete: remove them
find "$LOG_DIR" -type f -name "*.log" -mtime +$DAYS -delete
echo "Old logs cleared."
else
    echo "Directory $LOG_DIR not found."
fi
```

11.7 Global Search and Replace

This script searches for a specific string within all `.txt` files in a directory and replaces it with another string using `sed`.

```
#!/bin/bash

SEARCH_TERM="OLD_SERVER_IP"
REPLACE_TERM="NEW_SERVER_IP"

for file in *.txt; do
    if [[ -f "$file" ]]; then
        # -i: edit the file "in place"
        sed -i "s/$SEARCH_TERM/$REPLACE_TERM/g" "$file"
        echo "Updated $file"
    fi
done
```

11.8 Find Duplicate Files (Checksum Method)

Simply checking names isn't enough to find duplicates. This script uses MD5 Checksums to see if the actual content of two files is identical.

```
#!/bin/bash

# This script finds duplicate files in the current directory
declare -A file_hashes

for file in *; do
    if [[ -f "$file" ]]; then
        # Calculate MD5 hash
        hash=$(md5sum "$file" | awk '{print $1}')

        if [[ -n "${file_hashes[$hash]}" ]]; then
            echo "DUPLICATE FOUND: '$file' is the same as '${file_hashes[$hash]}'"
        else
            file_hashes[$hash]="$file"
        fi
    fi
fi
```

done

11.9 Large File Finder

This script identifies files larger than a user-defined size (e.g., 100MB) and lists them with their human-readable sizes.

```
#!/bin/bash

# Usage: ./large_files.sh /path/to/search 100M
SEARCH_PATH=${1:-.}
SIZE_LIMIT=${2:-50M}

echo "Searching for files larger than $SIZE_LIMIT in
    $SEARCH_PATH..."

# -size +100M: files larger than 100MB
# -exec ls -lh: execute 'ls' in human-readable format for each
# match
find "$SEARCH_PATH" -type f -size +"$SIZE_LIMIT" -exec ls -lh {}
+ | awk '{ print $5, $9 }'
```